AI Slop vs. Agentic Software Engineering

Same tools. Opposite outcomes. The difference is the human in the loop, not the model.

AI slop is code generated without understanding, design, or review, then shipped because it ran once. **Agentic SWE** is a partnership: a software engineer who owns the problem, working with one or more AI agents that draft, test, and critique inside a disciplined process. Left column is how slop gets made. Right column is how to do it properly.

	AI SLOP — how it gets made	AGENTIC SWE — how it should be made
STARTING POINT	✕ A vague prompt: “build me a login page.” No requirements, no constraints, no definition of done. ✕ The human has not thought about the problem before asking the AI to solve it.	✓ A written problem statement: who the user is, what must be true when it works, what is out of scope. ✓ The engineer owns the understanding. The agent is briefed like a new teammate, not a vending machine.
DESIGN	✕ None. The first thing the model emits becomes the architecture. ✕ Data model, API shape, and error handling are discovered by accident later.	✓ Design is discussed before code: options, trade-offs, a short architectural decision record (ADR). ✓ The agent proposes two or three approaches; the human picks and explains why. The chosen design is written down.
DOCUMENTATION	✕ No docs, or a README that was true three versions ago. The reasoning lives in a chat log nobody can find. ✕ Agents and new hires re-derive (and re-break) decisions because nothing explains why the code looks the way it does.	✓ Architecture and decision records live beside the code (e.g. /docs/architecture) and change in the same PR as the code. ✓ Each record captures the context, the options considered, and why one won. Agents read it before changing code; humans read it to onboard.
CONTEXT	✕ Model has no idea how the rest of the codebase works, so it reinvents helpers that already exist. ✕ Copy-pastes code into chat with no repo access.	✓ Agent works inside the repo with a written project guide: conventions, build and test commands, folder layout, and the “don’ts.” ✓ Existing patterns are read first and reused.
ITERATION	✕ One shot. Paste the output, run it, pray. ✕ When it fails, paste the stack trace and accept whatever comes back. Repeat until it stops erroring.	✓ Small, reviewable increments. Each step has a goal, a test, and a checkpoint commit. ✓ The engineer reads every diff and asks “why?” before moving on. Rejected ideas are rejected on purpose.
TESTING	✕ No tests, or tests generated after the fact that assert whatever the code happens to do. ✕ “It ran once on my machine” is the acceptance criterion.	✓ Tests express the intent, often written or agreed on before implementation. ✓ The agent runs the suite itself and reports real output. Failures are fixed, never skipped or deleted. ✓ Automated app tests (Playwright and similar) let the agent exhaustively exercise most or all user paths before review, which dramatically raises initial code quality.
STANDARDS	✕ Mixed naming, three styles of error handling, dead imports, commented-out code, giant files. ✕ Hallucinated APIs and dependencies that do not exist.	✓ Linters, formatters, and type checks run on every change and are enforced in CI. ✓ Every dependency is verified against its real docs. Style is the project’s style, not the model’s.
CODE REVIEW	✕ Nobody reads it. The author cannot explain it. “The AI wrote it” is offered as a reason, not a warning.	✓ The human reviews every line as if a junior had written it. If you can’t explain a line, it doesn’t ship. ✓ The author must be able to defend every design choice in the PR.

	AI SLOP — how it gets made	AGENTIC SWE — how it should be made
ADVERSARIAL REVIEW	✗ None. The same conversation that wrote the code says it looks great.	✓ A second agent, with fresh context, is told to break it: find bugs, security holes, missing edge cases, and unclear names. Use a reviewer from a different lab than the author (Codex + Claude, Gemini + Claude): different models have different blind spots. ✓ The human triages findings: confirmed issues become fixes or tickets; false positives are dismissed with a reason.
CODE SUBMISSIONS	✗ One 4,000-line commit titled "updates." No description, no test notes, pushed straight to main. ✗ Reviewers must reverse-engineer the intent. The PR sits for days, then gets rubber-stamped.	✓ Small PRs that state what, why, and how it was tested, plus a short video walkthrough or an auto-generated presentation of the change. ✓ Static analysis runs before a human looks: linting, formatting, and a quality gate (SonarQube-style) must pass. ✓ Reviewers focus on design and correctness, not style. Same-day answers to comments keep approval pace high.
SECURITY & SECRETS	✗ API keys in the prompt and in the code. Input validation is whatever the model felt like. ✗ Dependencies installed because the model named them.	✓ Secrets live in the environment, never in chat or source. Security review is an explicit step. ✓ Every new dependency is justified, pinned, and checked.
OWNERSHIP	✗ The human is a passenger. When it breaks in production, nobody understands it well enough to fix it.	✓ The human is the engineer of record. The agents are force multipliers. ✓ You are accountable for everything you ship, regardless of who or what typed it.

The Agentic SWE loop (one pass per small change)

1. **Understand** — write the problem, constraints, and acceptance criteria. Ask the agent to restate them.
2. **Design** — have the agent propose options; choose, and record the decision where the code lives.
3. **Plan** — break the work into small steps, each with a verifiable outcome.
4. **Implement** — agent writes code and tests in the repo, one step at a time, following project standards.
5. **Verify** — run unit and automated app tests, linters, formatters, and static analysis. Read the real output, not the summary.
6. **Review** — human reads the diff line by line. Ask the agent to explain anything unclear.
7. **Adversarial review** — a separate agent with fresh context attacks the change. Triage its findings.
8. **Document** — update the architecture docs and decision records in the same PR as the code.
9. **Submit** — small PR, clear description, walkthrough video or generated presentation, green CI, human approval and merge.

How to spot slop in a code review

- ✗ The author cannot explain why a line exists.
- ✗ Comments describe what the code does, never why.
- ✗ Functions and files that duplicate something already in the repo.
- ✗ Imports, packages, or API methods that do not exist.
- ✗ Tests that only assert the current behavior and never fail.
- ✗ Catch-all error handling that swallows exceptions silently.
- ✗ "The AI did it" offered as an explanation.

Anatomy of a great code submission

- ✓ Small and single-purpose: one change, one reason.
- ✓ Description answers what, why, and how it was tested.
- ✓ A short video walkthrough or auto-generated presentation.
- ✓ Static analysis already green: linting, formatting, quality gate.
- ✓ Architecture docs and decision records updated in the same PR.
- ✓ Adversarial review by an agent from a different lab already run; findings listed and resolved.
- ✓ Same-day replies to comments; fast submissions earn fast approvals.

The model is not the engineer. You are. Agents make a good engineer much faster and a careless one much more dangerous.